

CUADRO COMPARATIVO: PATRONES DE DISEÑO DE SOFTWARE

Cuadro Comparativo

Singleton	Creacional	Garantizar que una clase tenga únicamente una instancia en toda la aplicación y proveer un punto de acceso global a ella.	Restringir la creación de objetos de una clase a una sola instancia y controlar el acceso compartido.	Bajo	Gestión de conexiones a Base de Datos, Logger del sistema, Gestor de configuraciones globales, Spooler de impresión.
Factory Method	Creacional	Acoplamiento fuerte entre el código cliente y las clases concretas que se instancian directamente con 'new'.	Definir una interfaz para crear un objeto, pero dejar que las subclasses decidan qué clase concreta instanciar.	Medio	Creación de elementos de interfaz gráfica (UI) según el SO, procesadores de distintos formatos de archivo (PDF, CSV, JSON).
Abstract Factory	Creacional	Necesidad de crear familias de objetos relacionados o dependientes sin especificar sus clases concretas.	Proporcionar una interfaz para crear familias de objetos vinculados entre sí sin acoplar el código a sus clases concretas.	Alto	Sistemas multitema UI (botones, checkboxes, inputs estilo Windows/Mac/Linux), suites de renderizado gráfico 2D/3D.
Builder	Creacional	Construcción de objetos complejos que requieren múltiples parámetros o pasos de configuración (antipatrón 'constructor telescópico').	Separar la construcción de un objeto complejo de su representación, permitiendo crear diferentes representaciones con el mismo proceso.	Medio	Generadores de consultas SQL avanzadas, construcción de peticiones HTTP complejas, creación de documentos estructurados (PDF/HTML).
Adapter	Estructural	Incompatibilidad entre las interfaces de dos clases que necesitan colaborar juntas.	Convertir la interfaz de una clase en otra interfaz que el cliente espera, permitiendo que trabajen juntas clases incompatibles.	Bajo	Integración de librerías antiguas con código moderno, consumo de APIs externas con formatos de datos heterogéneos (XML a JSON).
Facade	Estructural	Complejidad alta al interactuar directamente con un	Proporcionar una interfaz unificada y simplificada para un	Bajo	Librerías de procesamiento multimedia

		subsistema compuesto por docenas de clases e interfaces.	conjunto de interfaces de un subsistema complejo.		(video/audio), sistemas bancarios complejos, motores de compresión de archivos.
Decorator	Estructural	Necesidad de añadir funcionalidades a objetos individuales dinámicamente sin afectar a otros objetos y sin recurrir a una explosión de subclases por herencia.	Añadir responsabilidades adicionales a un objeto dinámicamente envolviéndolo en clases decoradoras.	Medio	Streams de entrada/salida de datos (I/O en Java), agregación de toppings en pedidos de comida, middlewares de autenticación/compresión.
Observer	Comportamiento	Necesidad de notificar a múltiples objetos sobre los cambios de estado de otro objeto sin acoplar sus clases.	Definir una dependencia de uno a muchos entre objetos, de forma que cuando el objeto principal cambie de estado, todos sus dependientes sean notificados.	Bajo	Sistemas de eventos en interfaces gráficas (clicks), arquitectura Event-Driven, subscripciones a boletines o cotizaciones de bolsa.
Strategy	Comportamiento	Existencia de múltiples variantes de un algoritmo dentro de una misma clase, resultando en extensos condicionales (if/else o switch).	Definir una familia de algoritmos, encapsular cada uno de ellos y hacerlos intercambiables en tiempo de ejecución.	Medio	Algoritmos de ordenamiento, cálculo de rutas en mapas (a pie, auto, transporte público), estrategias de descuento o cálculo de impuestos.
Command	Comportamiento	Necesidad de parametrizar objetos con operaciones, cola de peticiones, almacenamiento de historial o soporte de operaciones deshacer (Undo/Redo).	Encapsular una petición/solicitud como un objeto independiente que contiene toda la información sobre la acción a realizar.	Medio	Sistemas de edición de texto/imágenes con ctrl+z, colas de procesamiento diferido (background jobs), automatización de tareas domésticas.

3. Análisis Detallado por Patrón de Diseño

Singleton (Creacional)

Objetivo Principal

Restringir la creación de objetos de una clase a una sola instancia y controlar el acceso compartido.

Problema que Resuelve	Garantizar que una clase tenga únicamente una instancia en toda la aplicación y proveer un punto de acceso global a ella.
Funcionamiento General	Utiliza un constructor privado y un método estático (ej. <code>getInstance()</code>) que crea la instancia la primera vez si no existe, o retorna la existente.
Ventajas	• Garantiza una única instancia. • Acceso global unificado. • Carga diferida (Lazy initialization).
Desventajas	• Viola el principio de responsabilidad única (SRP). • Dificulta las pruebas unitarias (estado global). • Puede requerir manejo especial en multihilo.
Casos de Uso	Gestión de conexiones a Base de Datos, Logger del sistema, Gestor de configuraciones globales, Spooler de impresión.
Ejemplo en Proyecto Real	En una aplicación web Spring Boot / Node.js, la gestión de un pool de conexiones a PostgreSQL para evitar agotar recursos creando cientos de conexiones simultáneas.
Nivel de Complejidad	Bajo

Factory Method (Creacional)

Objetivo Principal	Definir una interfaz para crear un objeto, pero dejar que las subclasses decidan qué clase concreta instanciar.
Problema que Resuelve	Acoplamiento fuerte entre el código cliente y las clases concretas que se instancian directamente con 'new'.
Funcionamiento General	Reemplaza las llamadas directas de construcción por un método fábrica abstracto o sobrescribible en subclasses.
Ventajas	• Evita acoplamiento con clases concretas. • Cumple el principio Abierto/Cerrado (OCP). • Facilita la extensión con nuevos productos.
Desventajas	• Puede incrementar la complejidad del código al requerir crear muchas subclasses para cada tipo de producto.
Casos de Uso	Creación de elementos de interfaz gráfica (UI) según el SO, procesadores de distintos formatos de archivo (PDF, CSV, JSON).
Ejemplo en Proyecto Real	Un sistema de pasarelas de pago que genera una instancia del procesador adecuado (PayPal, Stripe, MercadoPago) según el método seleccionado por el usuario.
Nivel de Complejidad	Medio

Abstract Factory (Creacional)

Objetivo Principal	Proporcionar una interfaz para crear familias de objetos vinculados entre sí sin acoplar el código a sus clases concretas.
Problema que Resuelve	Necesidad de crear familias de objetos relacionados o dependientes sin especificar sus clases concretas.
Funcionamiento General	Define interfaces para cada producto de la familia y una interfaz de fábrica abstracta con métodos para crear cada producto. Las fábricas concretas implementan estos métodos.
Ventajas	• Garantiza la compatibilidad entre objetos de la misma familia. • Aísla el código cliente de las clases concretas. • Facilita el reemplazo de familias completas.
Desventajas	• Complejidad elevada. • Dificultad para añadir nuevos tipos de productos a las familias existentes (requiere modificar todas las fábricas).
Casos de Uso	Sistemas multitema UI (botones, checkboxes, inputs estilo Windows/Mac/Linux), suites de renderizado gráfico 2D/3D.
Ejemplo en Proyecto Real	Un motor de videojuegos que genera interfaces y elementos gráficos compatibles para distintas plataformas (Xbox, PlayStation, PC).
Nivel de Complejidad	Alto

Builder (Creacional)

Objetivo Principal	Separar la construcción de un objeto complejo de su representación, permitiendo crear diferentes representaciones con el mismo proceso.
Problema que Resuelve	Construcción de objetos complejos que requieren múltiples parámetros o pasos de configuración (antipatrón 'constructor telescópico').
Funcionamiento General	Un objeto Builder acumula paso a paso la configuración del objeto deseado mediante métodos encadenables y finalmente produce el objeto con build().
Ventajas	• Permite construir objetos paso a paso o diferir pasos. • Reutiliza el código de construcción. • Mejora sustancialmente la legibilidad y previene constructores masivos.
Desventajas	• Aumenta la complejidad general del código al requerir crear clases Builder adicionales para cada clase compleja.
Casos de Uso	Generadores de consultas SQL avanzadas, construcción de peticiones HTTP complejas, creación de documentos estructurados (PDF/HTML).

Ejemplo en Proyecto Real	En Java con Lombok (@Builder) o en bibliotecas HTTP (como OkHttp/HttpClient) para configurar cabeceras, timeout, cuerpo y parámetros en una sola llamada fluida.
Nivel de Complejidad	Medio
Adapter (Estructural)	
Objetivo Principal	Convertir la interfaz de una clase en otra interfaz que el cliente espera, permitiendo que trabajen juntas clases incompatibles.
Problema que Resuelve	Incompatibilidad entre las interfaces de dos clases que necesitan colaborar juntas.
Funcionamiento General	Crea una clase adaptadora que implementa la interfaz esperada por el cliente y envuelve una instancia de la clase incompatible (adaptada).
Ventajas	• Cumple el principio de Responsabilidad Única y Abierto/Cerrado. • Permite reutilizar componentes existentes de legados o bibliotecas de terceros.
Desventajas	• Aumenta la complejidad del código al introducir interfaces y clases adicionales.
Casos de Uso	Integración de librerías antiguas con código moderno, consumo de APIs externas con formatos de datos heterogéneos (XML a JSON).
Ejemplo en Proyecto Real	Integrar un servicio de análisis analítico legacy que devuelve datos en formato XML en una plataforma moderna que solo procesa JSON.
Nivel de Complejidad	Bajo
Facade (Estructural)	
Objetivo Principal	Proporcionar una interfaz unificada y simplificada para un conjunto de interfaces de un subsistema complejo.
Problema que Resuelve	Complejidad alta al interactuar directamente con un subsistema compuesto por docenas de clases e interfaces.
Funcionamiento General	Una clase Fachada delega las peticiones de los clientes a los objetos apropiados dentro del subsistema, ocultando su complejidad interna.
Ventajas	• Aisla a los clientes de los componentes del subsistema. • Promueve un acoplamiento débil entre subsistemas y clientes.
Desventajas	• Una fachada puede convertirse en un objeto Dios (God Object) acoplado a todas las clases del subsistema si no se diseña con cuidado.
Casos de Uso	Librerías de procesamiento multimedia (video/audio), sistemas bancarios complejos, motores

de compresión de archivos.

Ejemplo en Proyecto Real Un módulo de pedidos en e-commerce que ofrece un método simple `realizarPedido()` que internamente coordina inventario, pago, facturación y notificación por email.

Nivel de Complejidad Bajo

Decorator (Estructural)

Objetivo Principal Añadir responsabilidades adicionales a un objeto dinámicamente envolviéndolo en clases decoradoras.

Problema que Resuelve Necesidad de añadir funcionalidades a objetos individuales dinámicamente sin afectar a otros objetos y sin recurrir a una explosión de subclases por herencia.

Funcionamiento General Tanto el objeto original como los decoradores comparten una interfaz común. El decorador envuelve el objeto base y añade su comportamiento antes o después de delegar la llamada.

Ventajas

- Gran flexibilidad para combinar comportamientos en tiempo de ejecución.
- Respeto el principio de Responsabilidad Única.

Desventajas

- Dificultad para eliminar un decorador específico de la pila de envolturas.
- Gran número de pequeños objetos similares.

Casos de Uso Streams de entrada/salida de datos (I/O en Java), agregación de toppings en pedidos de comida, middlewares de autenticación/compresión.

Ejemplo en Proyecto Real Java I/O con `BufferedReader(new InputStreamReader(new FileInputStream("file.txt")))`, donde cada clase añade una capa de funcionalidad.

Nivel de Complejidad Medio

Observer (Comportamiento)

Objetivo Principal Definir una dependencia de uno a muchos entre objetos, de forma que cuando el objeto principal cambie de estado, todos sus dependientes sean notificados.

Problema que Resuelve Necesidad de notificar a múltiples objetos sobre los cambios de estado de otro objeto sin acoplar sus clases.

Funcionamiento General El sujeto (Sujeto/Publisher) mantiene una lista de suscriptores (Observers) y provee métodos para suscribirse/describirse. Al cambiar su estado, recorre la lista notificándolos.

Ventajas

- Acoplamiento débil entre el sujeto y los observadores.
- Establece relaciones dinámicas en tiempo de ejecución.

Desventajas	• Los observadores son notificados en orden aleatorio y pueden causar actualizaciones en cascada innecesarias o costosas.
Casos de Uso	Sistemas de eventos en interfaces gráficas (clicks), arquitectura Event-Driven, subscripciones a boletines o cotizaciones de bolsa.
Ejemplo en Proyecto Real	Un sistema financiero donde la variación del precio de una acción notifica automáticamente al módulo de gráficos, al sistema de alertas por SMS y al registro de auditoría.
Nivel de Complejidad	Bajo

Strategy (Comportamiento)

Objetivo Principal	Definir una familia de algoritmos, encapsular cada uno de ellos y hacerlos intercambiables en tiempo de ejecución.
Problema que Resuelve	Existencia de múltiples variantes de un algoritmo dentro de una misma clase, resultando en extensos condicionales (if/else o switch).
Funcionamiento General	Extrae cada algoritmo en una clase separada que implemente una interfaz común (Estrategia). La clase contexto almacena una referencia a la estrategia actual.
Ventajas	• Aísla los detalles de implementación de un algoritmo. • Reemplaza herencia por composición. • Elimina extensas estructuras condicionales.
Desventajas	• Los clientes deben conocer las diferencias entre estrategias para elegir la adecuada en cada momento.
Casos de Uso	Algoritmos de ordenamiento, cálculo de rutas en mapas (a pie, auto, transporte público), estrategias de descuento o cálculo de impuestos.
Ejemplo en Proyecto Real	Una app de navegación como Google Maps que permite seleccionar la estrategia de cálculo de ruta dependiendo de si el usuario viaja caminando, en bicicleta o en automóvil.
Nivel de Complejidad	Medio

Command (Comportamiento)

Objetivo Principal	Encapsular una petición/solicitud como un objeto independiente que contiene toda la información sobre la acción a realizar.
Problema que Resuelve	Necesidad de parametrizar objetos con operaciones, cola de peticiones, almacenamiento de historial o soporte de operaciones deshacer (Undo/Redo).
Funcionamiento General	Convierte una acción y sus parámetros en un objeto Command con un método execute(). El

emisor ejecuta la orden sin conocer los detalles de la ejecución ni el receptor.

Ventajas

- Desacopla la clase que invoca la operación de la que sabe cómo realizarla.
- Facilita la implementación de Deshacer/Rehacer (Undo/Redo) y colas de tareas.

Desventajas

- Incrementa la cantidad de clases pequeñas en el código al crear una clase por cada orden específica.

Casos de Uso

Sistemas de edición de texto/imágenes con ctrl+z, colas de procesamiento diferido (background jobs), automatización de tareas domésticas.

Ejemplo en Proyecto Real

Un editor de imágenes como Photoshop o Figma donde cada acción del usuario (dibujar, mover, aplicar filtro) se almacena en una pila de objetos Command para revertir cambios.

Nivel de Complejidad

Medio

Referencias Bibliográficas

Refactoring.Guru. (2026). Patrones de diseño de software. Recuperado de <https://refactoring.guru/es/design-patterns>

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Oracle Java Documentation. (2026). Design Patterns in Java Platform. Oracle Docs.

Microsoft Learn. (2026). Software Design Patterns and Architecture Guidelines. Microsoft Technical Documentation.

PHP Documentation Group. (2026). Object-Oriented Programming and Patterns in PHP.